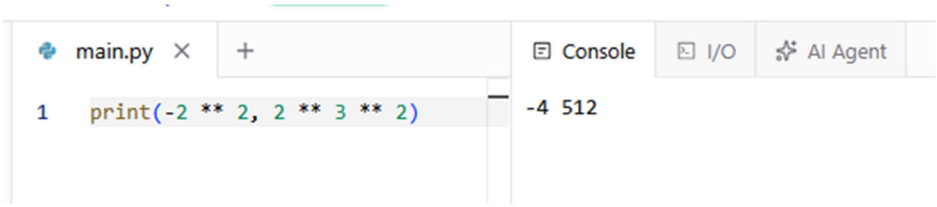
1 · Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

Palpite: acredito que será apresentado no terminal: **4 64.** Afirmo isso porque as operações matemáticas iniciam da esquerda para a direita. Portanto, a operação *-2 ** 2* retornaria *4*, enquanto *2 ** 3 (8) ** 2* retornaria *64*.



Validação: o resultado apresentado no terminal foi: **-4 512**. A primeira operação retornou *-4* devido à operação *2 ** 2* (potência) e somente após isso a multiplicação por *-1*. Além disso, a segunda operação retornou *512* por conta da potência entre os expoentes da base *2* (mais à direita), retornando *9* e sendo o resultado final: *2(9) = 512*.

2 · Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

Palpite: acredito que será apresentado no terminal: **erro de execução na concatenação entre texto e número.** Digo isso porque, em Java, uma operação aritmética de soma não é permitida entre valores que não são numéricos.



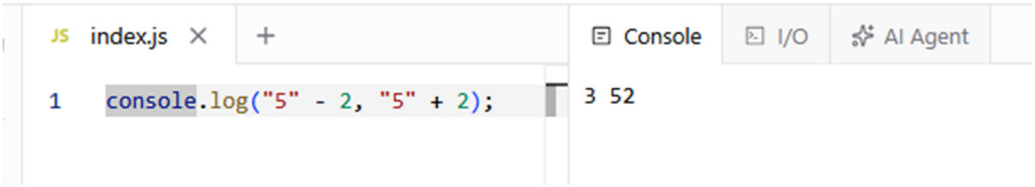
Validação: o resultado apresentado no terminal foi: **3345**. A primeira (1ª) operação de soma entre *1* e *2* ocorreu corretamente. Porém, ao identificar que o terceiro (3º) elemento era textual, concatenou o valor *"3"* ao resultado da soma. Por fim, entendendo que os demais elementos

eram textuais, devido ao terceiro (3º) valor, concatenou-os juntamente com o resultado até aquele momento.

3 · JavaScript

```
console.log("5" - 2, "5" + 2);
```

Palpite: acredito que o resultado apresentado no terminal será: **3 52**. Afirmo isso porque a primeira operação será interpretada como uma subtração entre os números *5* e *2*. Porém, a segunda operação será entendida como uma concatenação.

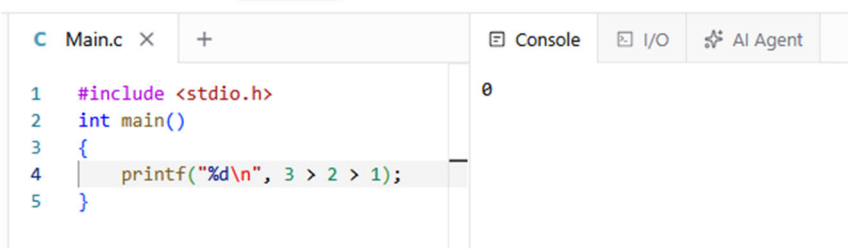


Validação: o resultado apresentado foi: **3 52.** O motivo foi que o operador *+* tem mais de uma função dentro da linguagem Javascript (soma/concatenação). Enquanto isso, o operador – tem uma única função.

4 · C

```
printf("%d\n", 3 > 2 > 1);
```

Palpite: acredito que o resultado apresentando no terminal será: **1**. Entendo que a expressão lógica *3 > 2 > 1* é verdadeira, portanto será retornado o valor *1* (representação *booleana* de verdadeiro em C).

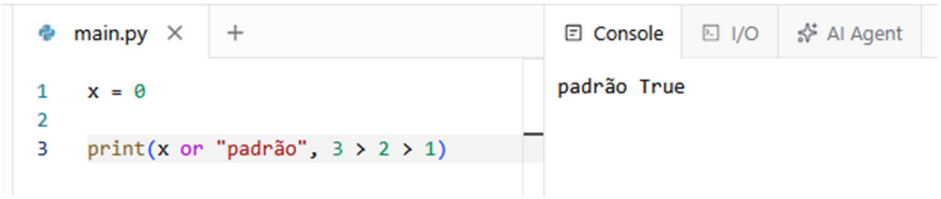


Validação: o resultado apresentado foi: 0. O motivo foi que a expressão *booleana* foi fragmentada em duas (2) partes: a primeira (1ª) *3 > 2* e a segunda (2ª) 1 (verdadeiro) *> 1*. A primeira expressão é correta, enquanto a segunda (*1 > 1*) é falsa. Portanto, o resultado é 0 (representação em C de *falso*).

5 · Python

```
x = 0
print(x or "padrão", 3 > 2 > 1)
```

Palpite: acredito que o resultado apresentado no terminal será: **True True**. Digo isso pois a primeira expressão retornará verdadeiro caso algum dos termos não seja vazio e/ou falso. A segunda expressão, retornará o resultado das condições aninhadas *3 > 2* e *2 > 1* = *True*.

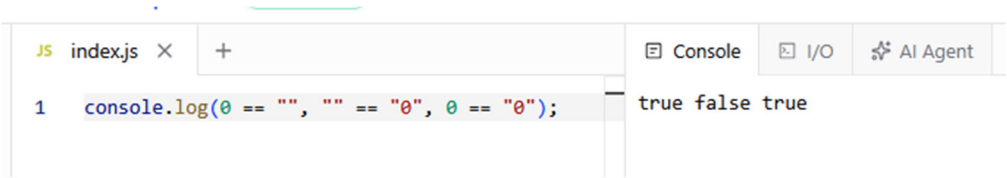


Validação: o resultado apresentado foi: **padrão True**. O motivo foi que a primeira expressão retornou o primeiro termo não nulo da expressão, portanto o texto à direita. Além disso, a segunda expressão realmente aplicou duas (2) condições lógicas aninhadas: *3 > 2* e *2 > 1*.

6 · JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

Palpite: acredito que o resultado apresentado no terminal será: **true, false, true**. Digo isso porque a primeira expressão compara *0* (representativo de vazio) com um texto vazio, enquanto a segunda expressão avalia um texto vazio diante de um texto preenchido. Por fim, a última expressão compara o valor numérico *0* com o texto numérico *0* (que por sua vez é interpretado como número).



Validação: o resultado apresentado foi o mesmo registrado acima: **true, false, true**. A primeira e terceira (1ª e 3ª) expressão comparou termos com o mesmo significado. Enquanto a segunda (2ª) expressão avaliou termos semanticamente diferentes.